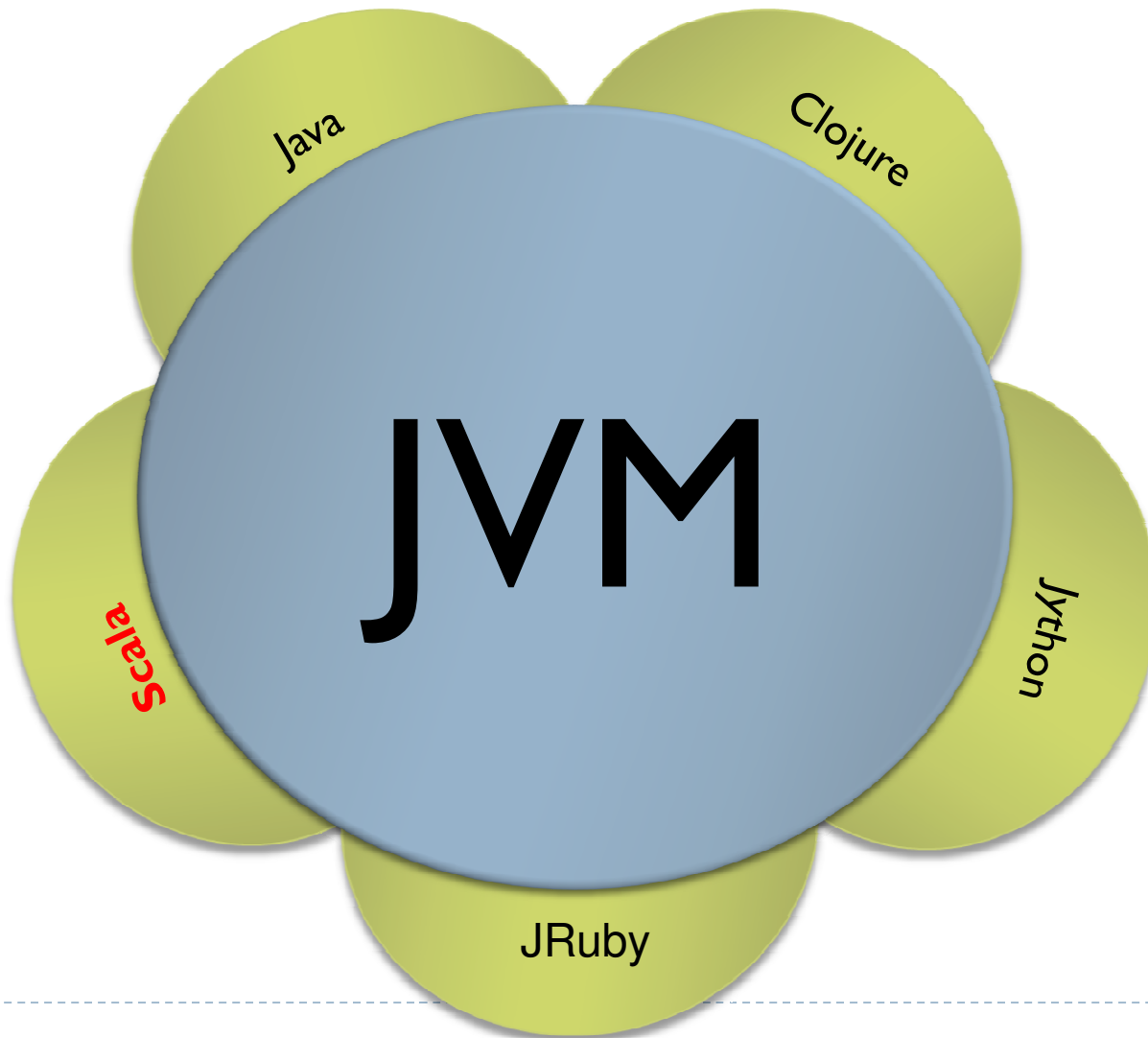


# Concepts des langages de programmation

Introduction à Scala

# JVM (*Java Virtual Machine*)

---



# Scala

---

- ▶ Développé en 2001 par [Martin Odersky](#) à l'École Polytechnique Fédérale de Lausanne (EPFL) en suisse
- ▶ Première version publique sortie fin 2003
  - ▶ Version JVM
  - ▶ Version .NET mi 2004
- ▶ Utilisé entre autres par
  - ▶ Twitter (remplacement de quelques modules écrits en Ruby)
  - ▶ EDF

# Scala (suite)

---

- ▶ Scala **est fonctionnel**
- ▶ Scala **est orienté objet**
  - ▶ Tout est objet
- ▶ Scala **est concurrent**
  - ▶ Offre la bibliothèque des *Actors* : communication par messages pour la parallélisation de tâches pour s'exécuter sur plusieurs noyaux (multi-core)

# Fonctionnalités de Scala

---

- ▶ Scala est à typage statique
  - ▶ Inclus un **système d'inférence** de types locaux :

- ▶ Dans Java 1.5 :

```
Pair p = new Pair<Integer, String>(1, "Scala");
```

- ▶ Dans Scala :

```
val p = new MyPair(1, "scala");
```

## Autres fonctionnalités (suite)

---

- ▶ Toute fonction peut être utilisée en tant qu'opérateur infixe ou postfixe
- ▶ Permet de définir des méthodes avec des noms tel que :  $+$ ,  $<$ ,  $=$ , ...

# Commentaires en Scala

---

- ▶ Les commentaires se définissent de la même façon qu'en Java :

- ▶ `/*`

- Ceci est un commentaire

- Sur plusieurs lignes

- `*/`

- ▶ `//` En voici un autre commentaire mais sur une seule ligne

# Déclaration de variables et mécanisme d'inférence

---

- ▶ La déclaration d'une variable se fait de la manière suivante
  - ▶ **var** nomVariable:**Type**
  - ▶ **val** nomVariable:**Type**
- ▶ Exemple :
  - ▶ var maChaine:**String** = 'Salut tout le monde !'
- ▶ Grace au mécanisme d'inférence de Scala la déclaration et affectation ci-dessus peut être écrite comme suit :
  - ▶ var maChaine = 'Salut tout le monde !'



# Utilisation de **val** à la place de **var**

---

- ▶ **var** s'utilise pour déclarer des objets mutables
  - ▶ La valeur attribuée peut être modifiée
  - ▶ `var s = "Salut" ; s = "comment ça va ?"`
- ▶ **val** s'utilise pour déclarer des objets non mutables ('constantes')
  - ▶ La valeur ne peut être modifiée
  - ▶ ~~`val s = "Salut" ; s = "comment ça va ?"`~~
- ▶ Remarque :
  - ▶ Les propriétés de l'objet référencé par **val** peuvent être modifiées (exemple : éléments d'un tableau) et la référence ne peut pas être modifiée

# Types de données

---

## ▶ Quelques types

- ▶ **Byte** : entiers signés de 8-bits
- ▶ **Short** : entiers signés de 16-bits
- ▶ **Int** : entiers signés de 32-bits
- ▶ **Long** : entiers signés de 64-bits
- ▶ **Float** : réels sur 32-bits
- ▶ **Double** : réels 64-bits
- ▶ **Boolean** : true or false
- ▶ **BigInt** : très grands entiers
- ▶ **Char** : caractère unicode de 16-bits
- ▶ **String** : une suite de caractères

- ▶ Par convention, les noms des types de données commencent par une majuscule
- ▶ La version en minuscule peut être utilisée mais elle fera référence tout de même à la version majuscule du même type
- ▶ Exemple : `int` fait référence à `Int` qui à son tour fait référence à `scala.Int`

# Tableaux

---

- ▶ `val tab = new Array[String](3)`
- ▶ Une fois déclaré, la taille d'un tableau ne peut pas être changée
- ▶ Les éléments d'un tableau sont tous de même type
- ▶ Les éléments d'un tableau sont **mutables**
  - ▶ `tab(0) = "Bonjour"`
  - ▶ `tab(1) = "Hello"`
  - ▶ `tab(2) = "Guten Tag"`
  - ▶ `// équivalent à Array(" Bonjour" ,"Hello","Guten Tag")`
- ▶ Itérer sur les éléments d'un tableau
  - ▶ `for (i <- 0 to 2) print(tab(i)) // Bonjour Hello Guten Tag`

# Listes

---

- ▶ Deux manières différentes d'initialiser une liste
  - ▶ `val u = 1 :: 2 :: 3 :: 4 :: 5 :: Nil`
  - ▶ `val t = List( 1, 2, 3, 4, 5)`
- ▶ Possèdent l'opérateur de concaténation '::
  - ▶ `val oneTwo = List(1, 2)`
  - ▶ `val threeFour = List(3, 4)`
  - ▶ `val oneTwoThreeFour = oneTwo ::: threeFour // List(1, 2, 3, 4)`
  - ▶ `val oneTwoThree = 1 :: 2 :: 3 :: Nil // List(1, 2, 3)`
- ▶ Les éléments d'une liste sont tous de même type
- ▶ Les listes sont **immuables**

# Tuples (enregistrements)

---

- ▶ `val paire = (23, "rouge")`
- ▶ Les éléments sont accédés avec l'opérateur `'._'`
  - ▶ `println(paire._1) // 23`
  - ▶ `println(paire._2) // rouge`
- ▶ Les éléments d'un tuple peuvent être de types différents
- ▶ Très utilisés pour qu'une méthode puisse retourner des objets de types différents
- ▶ Les tuples sont des objets **immutables**

# Tables de hachage

---

- ▶ Il existe deux types de tables de hachage :
  - ▶ Tables de hachage mutables
  - ▶ Tables de hachage immutables

# Tables de hachage mutables

---

- ▶ `import scala.collection.mutable.HashMap`
- ▶ `val tabHachage = new HashMap[Int, String]`
- ▶ `tabHachage += 1 -> "Salut"`
- ▶ `tabHachage += 2 -> "Bonjour"`
- ▶ `tabHachage += 3 -> "Bonsoir"`
- ▶ `println(tabHachage(2))`

# Tables de hachage immutables

---

- ▶ `import scala.collection.immutable.HashMap`
- ▶ `val immutable = HashMap (1 -> "one", 2 -> "two", 3 -> "three")`



# Instructions conditionnelle if ... else

---

- ▶ Similaire à la déclaration du if ... else en Java

- ▶ `if(condition){`
    - ▶ `//bloc d'instructions 1`
  - ▶ `}`
  - ▶ `else{`
    - ▶ `//bloc d'instructions 2`
  - ▶ `}`

- ▶ Exemple :

- ▶ `val l = l`
  - ▶ `if(i < 0){`
    - ▶ `println("nombre négatif")`
  - ▶ `}else{`
    - ▶ `println("nombre positif ou nul")`
  - ▶ `}`

# Boucles while

---

- ▶ `while(condition){`
  - ▶ `//instructions`
- ▶ `}`

- ▶ **Exemple :**

- ▶ `var i = 0`
- ▶ `while (i < 10000000) {`
  - ▶ `// instructions`
  - ▶ `i = i+1`
- ▶ `}`

# Boucles for

---

## ► Trois manières d'écrire une boucle for :

1. for (i <- 0 **until** 10) { // le 10 n'est pas inclus (10 itérations)

► //instructions

► }

2. for (i <- 0 **to** 10) { // le 10 est inclus (11 itérations)

► //instructions

► }

3. for (i <- List(5, 7, 2, 4)) { // ou for (i <- Array(5, 7, 2, 4))

► //instructions

► }

## ► Remarque :

► Le symbole '<-' se lit : *in* (dans)

# Choix multiple

---

## ► Choix multiples dans **Scala** :

```
► expr match {  
  case pati => expri  
  ....  
  case patn => exprn  
  case _      => expr_par_default  
}
```

## ► Choix multiples dans **Java** :

```
► switch (expr) {  
  case pati : return expri;  
  ...  
  case patn : return exprn;  
  default: expr_par_default;  
}
```

# Fonctions

---

► Définition de fonctions en **Scala** :

► **def** fonction (x: Int): Int = {  
    // code  
    result  
}

► Ou bien :

► **def** fonction(x: Int)= result

► Définition de fonctions en **Java** :

► **int** fonction(**int** x) {  
    // code  
    **return** result  
}

► (pas d'équivalent)

# Fonctions (suite)

---

- ▶ Appel de fonction dans **Scala** :
  - ▶ **obj.fonction(arg)**
  - ▶ **obj fonction arg**

- ▶ Appel de fonction dans **Java** :
  - ▶ **obj.fonction(arg)**
  - ▶ (pas d'équivalent)

# Fonctions (suite)

---

- ▶ Les types des paramètres doivent être définis explicitement
- ▶ Lorsque la fonction ne retourne pas de valeurs, alors on utilise le mot clé *Unit* (similaire au void de java)

```
def max(x: Int, y: Int): Int = {  
    if (x > y) x else y  
}  
  
// Function sans type de retour  
def echo(msg: String) : Unit = {  
    println(msg)  
}
```

# Procédures

---

- ▶ Une fonction qui ne retourne pas de valeur peut être écrite dans un format spécial : **les procédures**
  - ▶ Pas de type de retour
  - ▶ Pas de symbole '=' après la définition de la fonction
  - ▶ **La version procédurale est implicitement convertie par le compilateur au format standard des fonctions (en ajoutant le mot clé *Unit*)**

```
// version procédurale de la fonction echo ()  
  
def pecho(msg: String) {  
    println(msg);  
}
```



# Fonctions anonymes

---

- ▶ Ce sont des fonctions qui **ne sont pas nommées**
- ▶ Syntaxe :
  - ▶ **(arguments) => corps de la fonction**
- ▶ Exemple :
  - ▶ **(chaine: String) => println(chaine)**
  - ▶ Ci-dessus est la déclaration d'une fonction anonyme qui admet en paramètre une chaîne de caractères et dont le corps est une instruction d'affichage (println())

# Fonctions anonymes (suite)

---

- ▶ Exemple d'utilisation
  - ▶ `tab.foreach ((chaine: String) => println(chaine))`
- ▶ Ceci est équivalent à :
  - ▶ `tab.foreach (chaine => println(chaine))`
- ▶ Résultat de l'exécution :
  - ▶ Bonjour
  - ▶ Hello
  - ▶ Guten Tag

# Ombrage (*shadowing*)

---

- ▶ Contrairement à Java, Scala permet de **réutiliser les noms de variables à l'intérieur des block imbriquées**
- ▶ Ceci est appelé de l'ombrage

```
def shadow() {  
    val a = 1;  
    {  
        val a = 2;  
        println("a à l'intérieur du bloc : " + a); // affichera 2  
    }  
    println("a déclaré avant le bloc ci-dessus: " + a); // affichera 1  
}
```

---

# Objets et classes

# Objets et classes

---

Classe et object en **Scala** :

```
▶ class Exemple (var x: Int, val p: Int)
{
  def instMeth(y: Int) = x + y
}
```

// des getters sont créés automatiquement  
pour x et p

// un setter est créé pour x et non pour p

Instantiation

```
▶ val g = new Exemple
```

Classe **Java** :

```
▶ class Exemple {
  public final int x;
  private final int p;
  Exemple (int x, int p) {
    this.x = x;
    this.p = p;
  }
  int instMeth(int y) {
    return x + y;
  }
}
```

Instantiation

```
▶ Exemple g = new Exemple
```

# Objets et classes (suite)

Méthodes statiques en **Scala** :

```
▶ class Exemple (x: Int, val p: Int) {  
    def instMeth(y: Int) = x + y  
}  
  
object Exemple {  
    def staticMeth(x: Int, y: Int) =  
        x * y  
}
```

**object** est utilisé pour définir une classe avec une instance unique (*singleton*)

Méthodes statiques **Java** :

```
▶ class Exemple {  
    private final int x;  
    public final int p;  
    Exemple (int x, int p) {  
        this.x = x;  
        this.p = p;  
    }  
    int instMeth(int y) {  
        return x + y;  
    }  
    static int staticMeth(int x, int y) {  
        return x * y;  
    }  
}
```

# Objets et classes (suite)

Constructeurs auxiliaires en **Scala** :

```
► class Exemple (x: Int, val p: Int) {  
    def instMeth(y: Int) = x + y  
    def this(a: String) = println(a)  
}
```

Constructeurs auxiliaires en **Java** :

```
► class Exemple {  
    private final int x;  
    public final int p;  
    public int a;  
    Exemple (int x, int p) {  
        this.x = x;  
        this.p = p;  
    }  
    Exemple (String a) {  
        System.out.println(a);  
    }  
    int instMeth(int y) {  
        return x + y;  
    }  
}
```

# Objets et classes (suite)

Méthode main en **Scala** :

```
▶ class Exemple (x: Int, val p: Int) {  
    def instMeth(y: Int) = x + y  
    def main(args :Array[String]) : Unit = {  
        // instructions  
    }  
}
```

Méthode main en **Java** :

```
▶ class Exemple {  
    private final int x;  
    public final int p;  
    public int a;  
    Exemple (int x, int p) {  
        this.x = x;  
        this.p = p;  
    }  
    public static void main (String[]  
args) {  
        // instructions  
    }  
    int instMeth(int y) {  
        return x + y;  
    }  
}
```



# Polymorphisme et héritage

---

- ▶ Héritage

- ▶ Les classes peuvent hériter/étendre d'autres classes

- ▶ Polymorphisme

- ▶ Les méthodes peuvent être surchargées
    - ▶ Même nom mais **nombre de paramètres différents** ou de **types différents**
  - ▶ Les méthodes peuvent être redéfinies
    - ▶ **Réécrire des méthodes déjà existantes** au niveau des surclasses qui ont les mêmes signatures

# Traits

---

- ▶ Ce sont comme des classes
  - ▶ Un moyen d'encapsuler des méthodes et des données
- ▶ Dans Scala les traits ne s'intègrent pas avec l'héritage ce sont par contre des “mixed-in”
  - ▶ Les mixins (contraction de *Mixed-in*) est un moyen de mixer du code à l'intérieur de notre classe/objet pour qu'il en devient une partie de la classe/objet
  - ▶ Contrairement à l'héritage, où chaque classe hérite d'une seule superclasse, une classe peut mixer un nombre variable (non limité) de Traits
  - ▶ Les Traits de Scala sont l'équivalent des modules de Ruby où ce qui ressemble aussi aux interfaces en Java

# Exemple de traits

---

```
01 trait Philosophical {  
02     def philosophize() {  
03         println("Je consomme de la mémoire donc j'existe !")  
04     }  
05 }  
06 class Mac1 extends Philosophical { }  
07 class Mac2 {}  
08 scala> val air1 = new Mac1  
  
09 scala> val air2 = new Mac2 with Philosophical  
  
09 scala> air1.philosophize()  
10 Je consomme de la mémoire donc j'existe !  
  
11 scala> air2.philosophize()  
12 Je consomme de la mémoire donc j'existe !
```

# Évaluation paresseuse

---

- ▶ On utilise le mot clé : **lazy**
- ▶ Exemple illustratif de l'évaluation paresseuse

```
▶ object Exemple {  
  val x = {println("initialisation de x"); "faite"}  
}
```

```
scala> Exemple  
initialisation de x  
res1: Exemple.type = Exemple$@148e23b  
scala>
```

```
▶ object Exemple {  
  lazy val x = {println("initialisation de x"); "faite"}  
}
```

```
scala> Exemple  
res0: Exemple.type = Exemple$@1eb2473  
scala> Exemple.x  
initialisation de x  
res1: java.lang.String = faite  
scala> _
```

# Quelques concepts pratiques

---

## ► Map

- `List(1,2,3).map(_+10).foreach(println)`
- `// afficher chaque élément augmenté de 10`
- `_` : fait référence à un élément de la liste

## ► Filtrage (et intervalles)

- `(1 to 100) filter (_ % 7 == 3) foreach (println)`
- `// afficher les éléments allant de 0 à 100 dont le reste`  
`//de la division par 7 est égal à 3`



## Quelques concepts pratiques (suite)

---

- ▶ Initialisation par défaut

- ▶ `var i : Int = _` // équivalent à : `var i : Int = 0`

- ▶ `var chaine : String = _` // équivalent à : `var chaine : String = ""`



# Résumé : concision

---

// Java

```
public class Person {  
    private String name;  
    private int age;  
    public Person(String name,  
                   int age) {  
        this.name = name;  
        this.age = age;  
    }  
    public String getName() {  
        return name;  
    }  
    public int getAge() {  
        return age;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public void setAge(age: int) {  
        this.age = age;  
    }  
}
```

// Scala

```
class Person(  
    var name: String,  
    var age: Int)
```

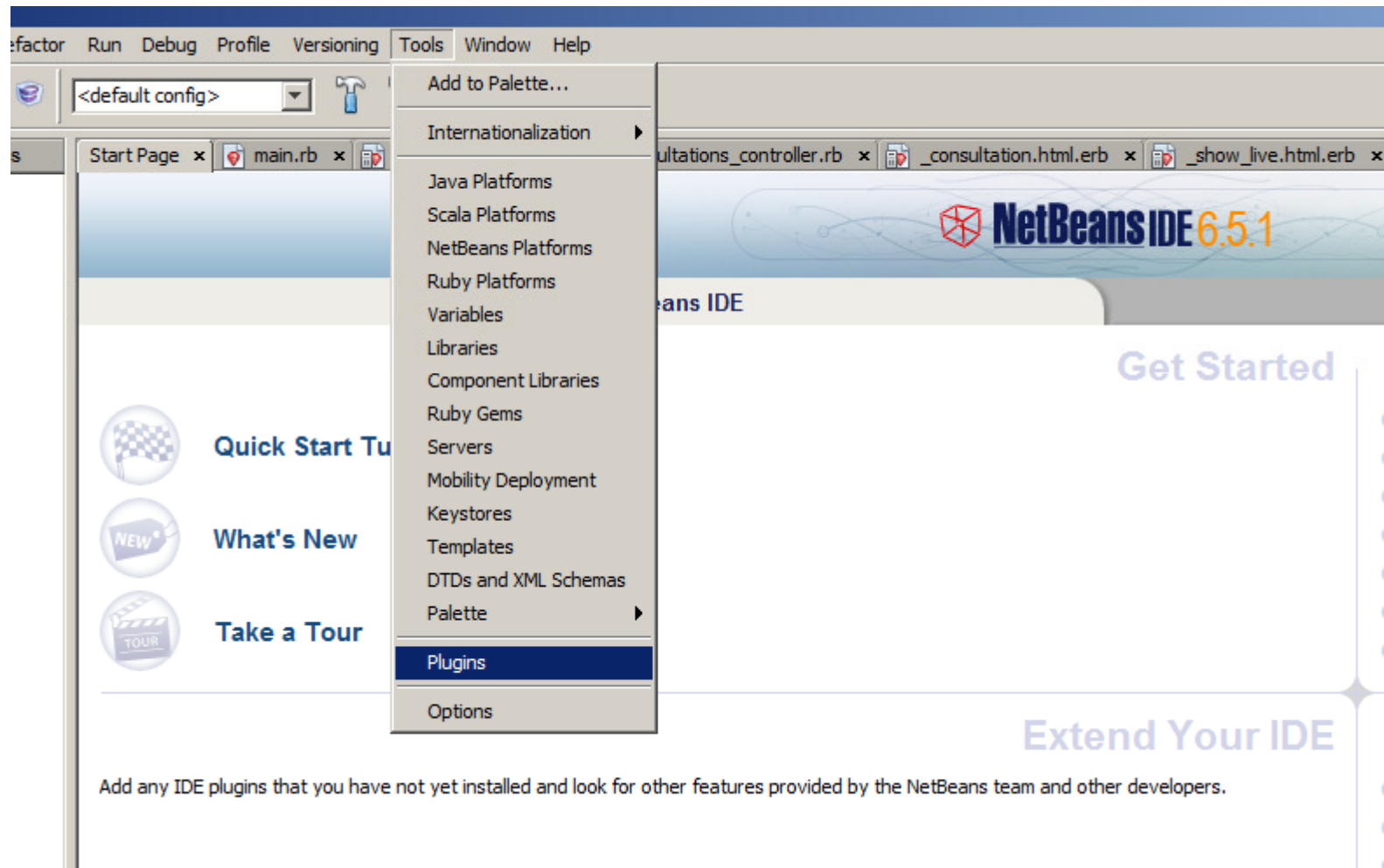
# Scala et Netbeans

---

- ▶ Un plugin Netbeans pour programmer avec Scala est disponible
- ▶ Installation du plugin :
  - ▶ Télécharger le plugin Netbeans pour Scala de la page Web du cours ou de [http://sourceforge.net/project/showfiles.php?group\\_id=192439&package\\_id=256544](http://sourceforge.net/project/showfiles.php?group_id=192439&package_id=256544)
  - ▶ Décompresser l'archive
  - ▶ Suivre les étapes indiquées par les captures d'écran des diapositifs qui suivent

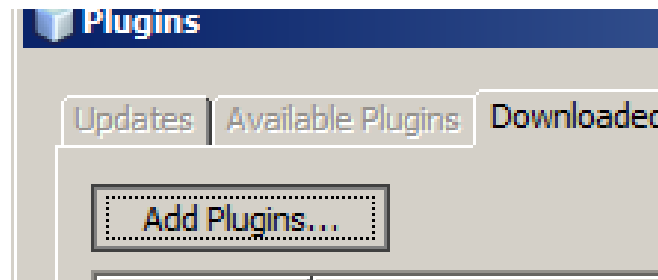


# Installation du plugin

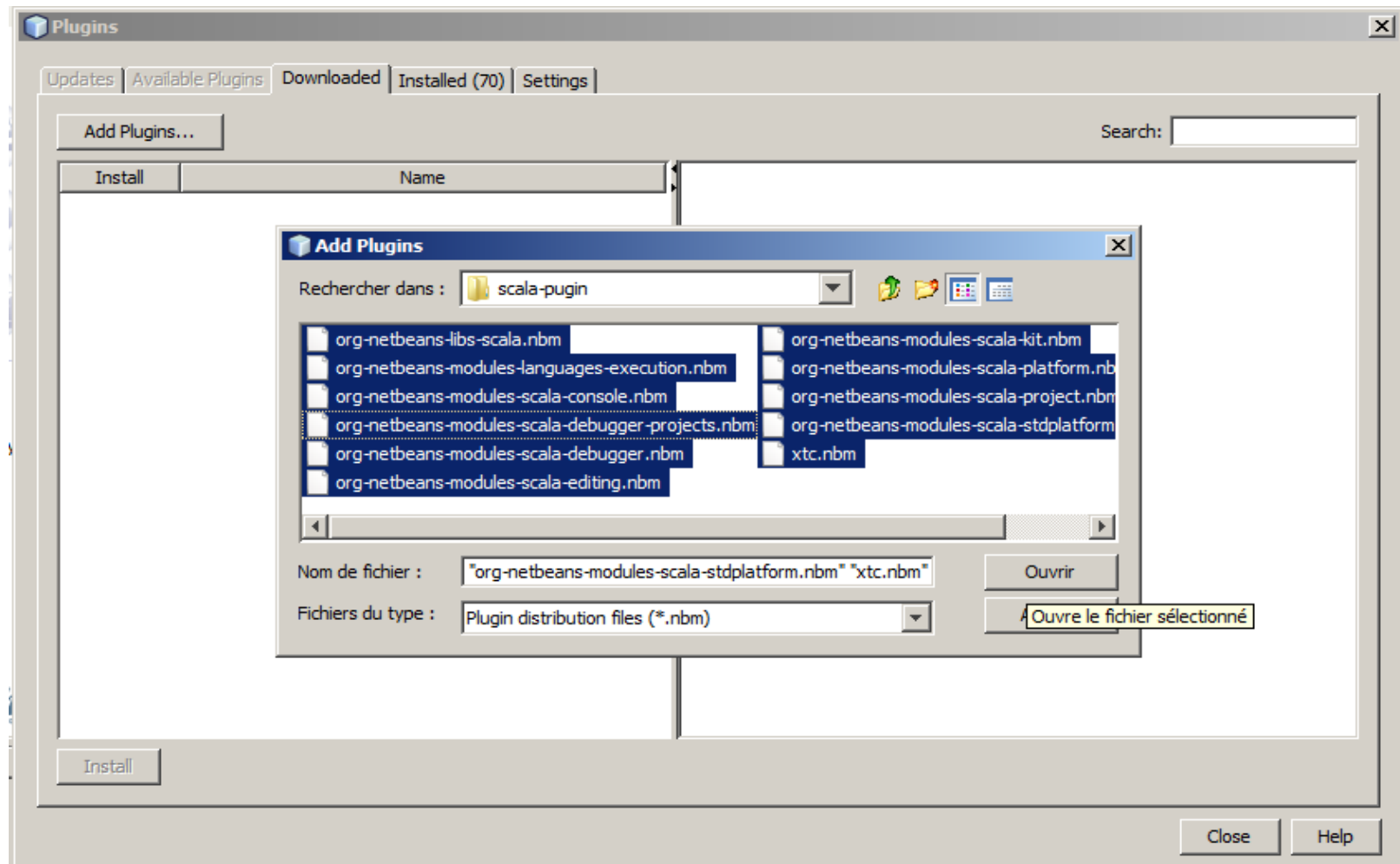


# Installation du plugin (suite)

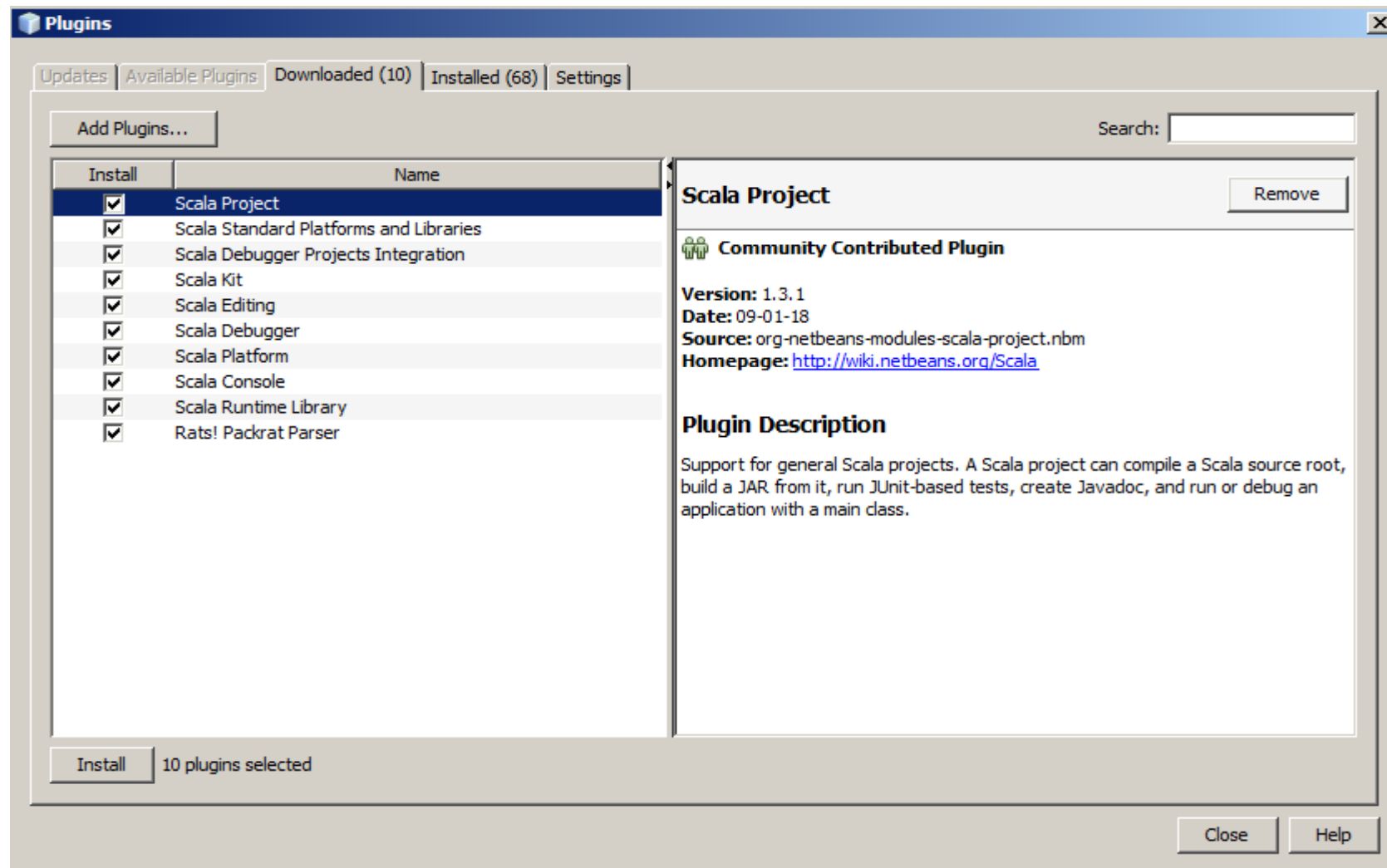
---



# Installation du plugin (suite)

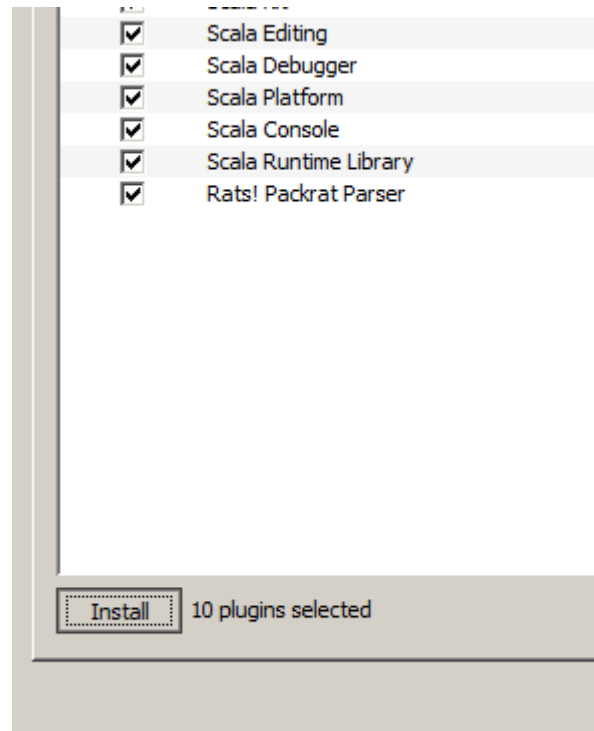


# Installation du plugin (suite)



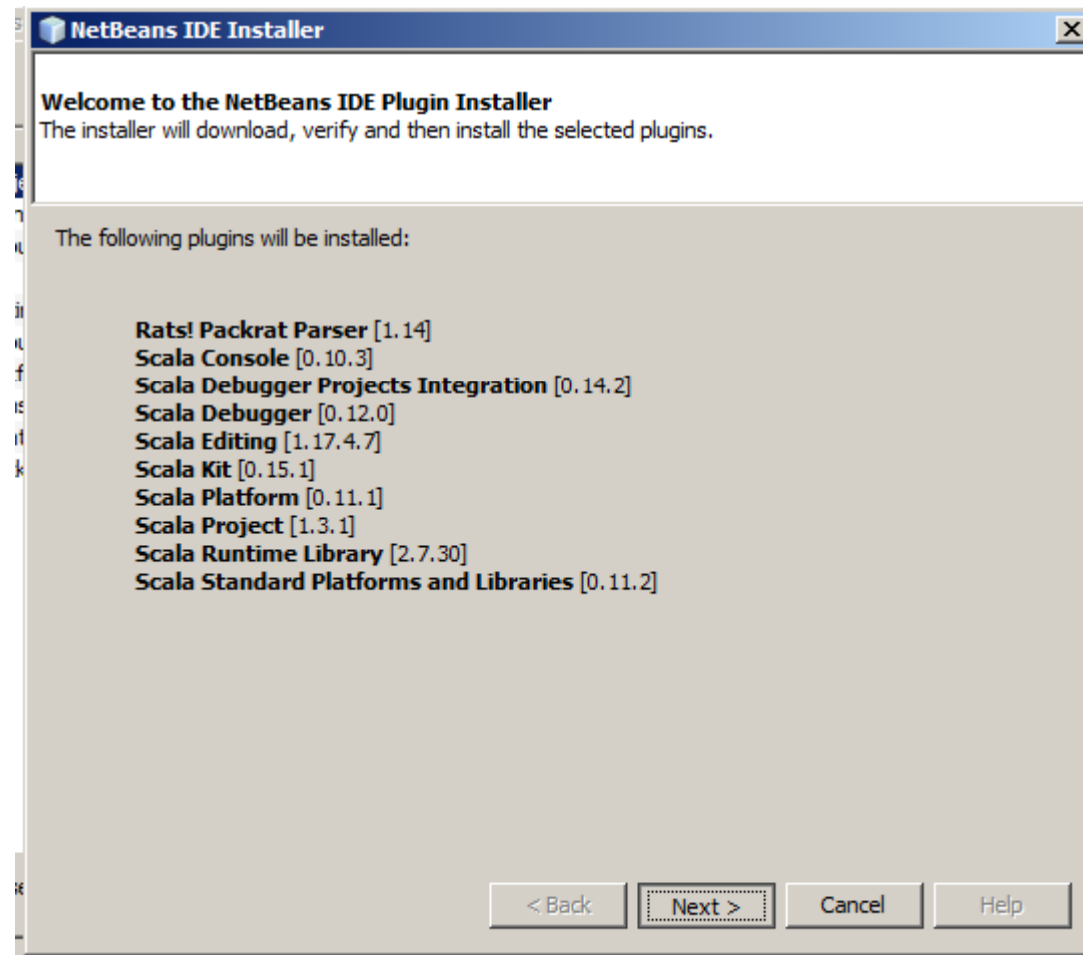
# Installation du plugin (suite)

---

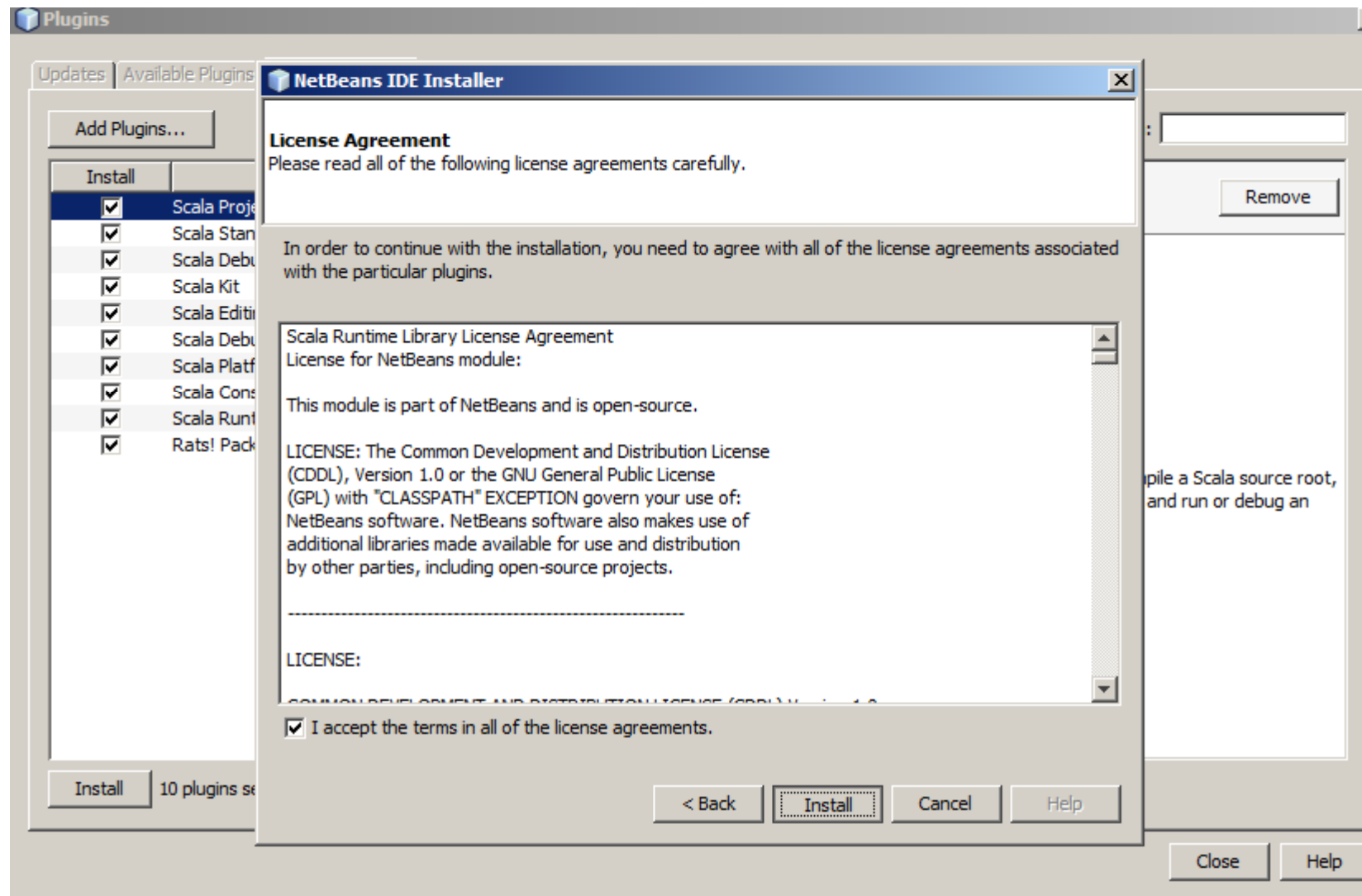


# Installation du plugin (suite)

---

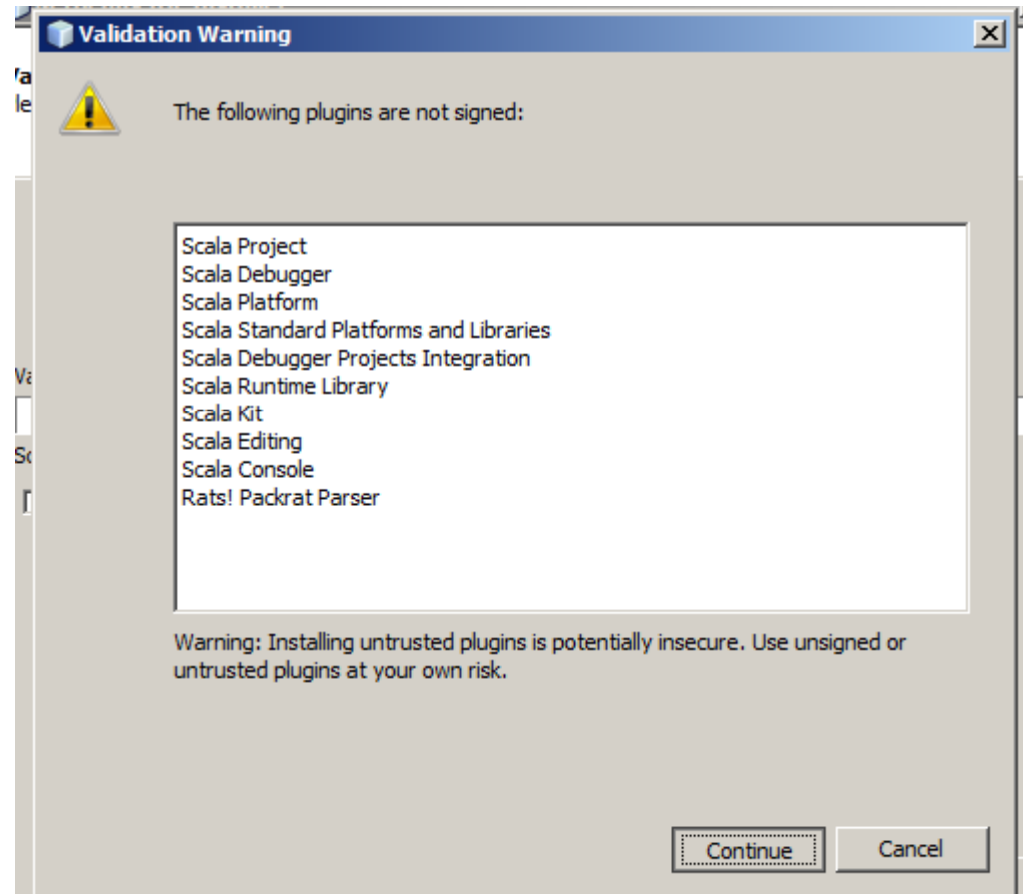


# Installation du plugin (suite)



# Installation du plugin (suite)

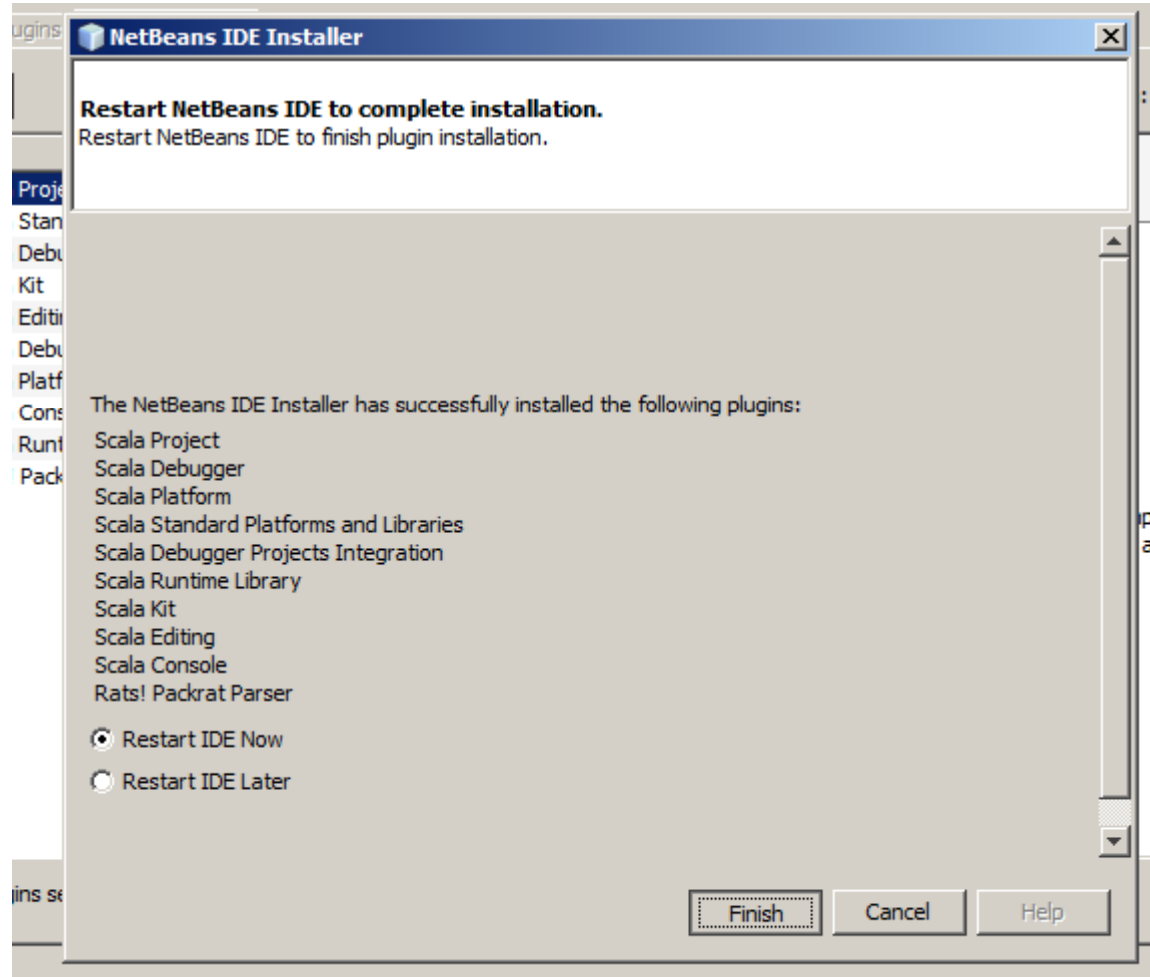
---



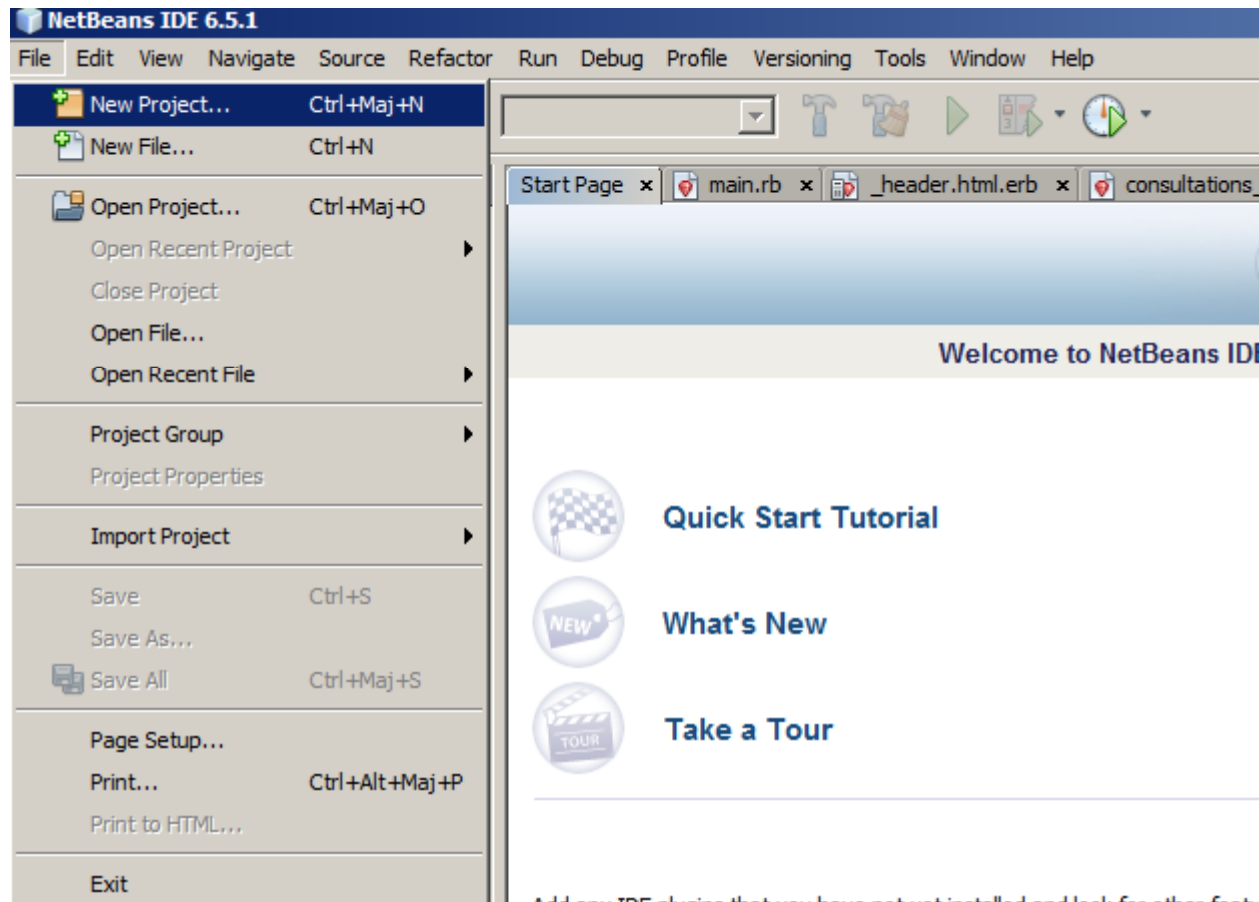


# Installation du plugin (suite)

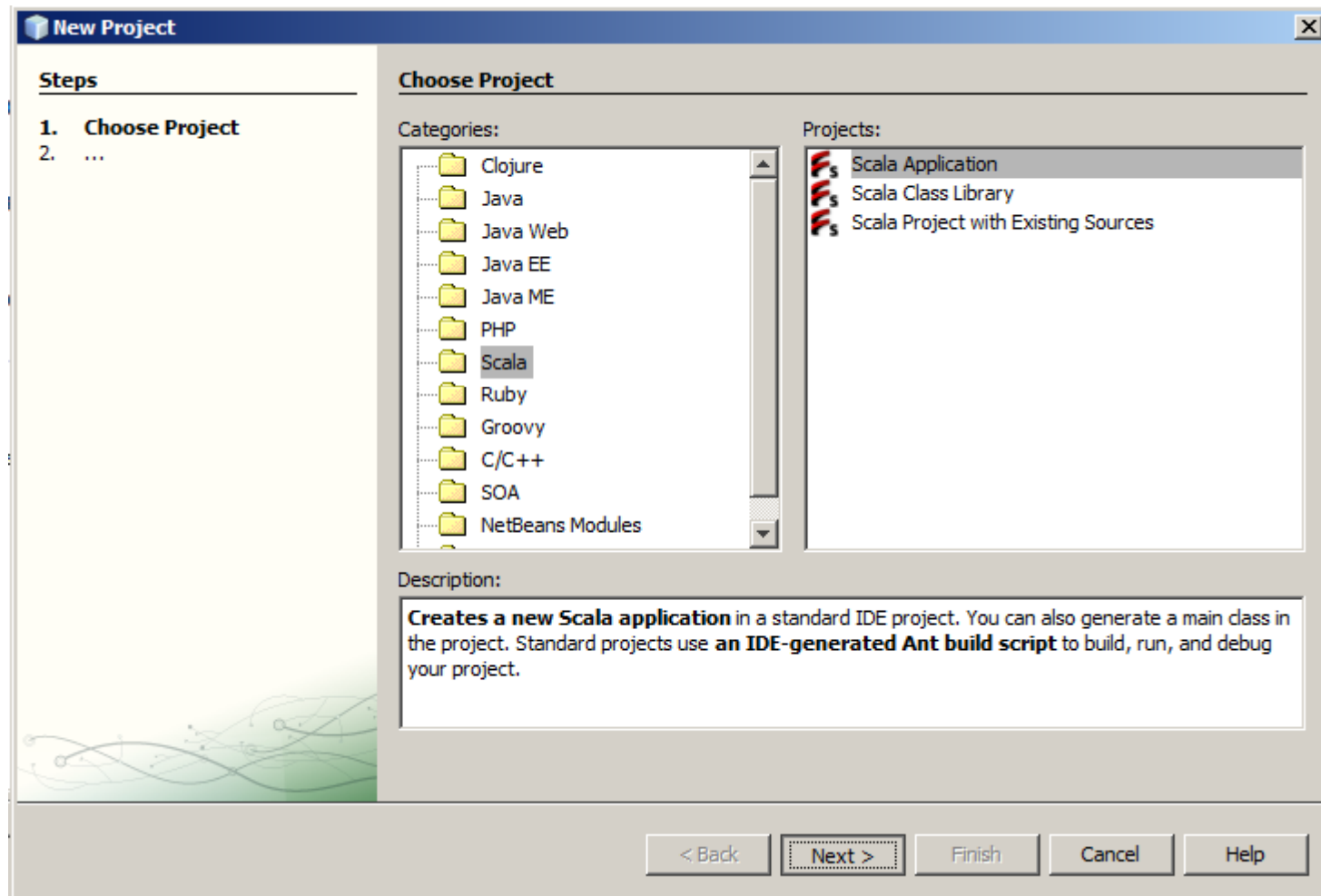
---



# Création d'un nouveau projet Scala



# Création d'un nouveau projet Scala (suite)



# Création d'un nouveau projet Scala (suite)

**New Scala Application**

**Steps**

1. Choose Project
2. **Name and Location**

**Name and Location**

Project Name:

Project Location:

Project Folder:

☐ Use Dedicated Folder for Storing Libraries

Libraries Folder:

Different users and projects can share the same compilation libraries (see Help for details).

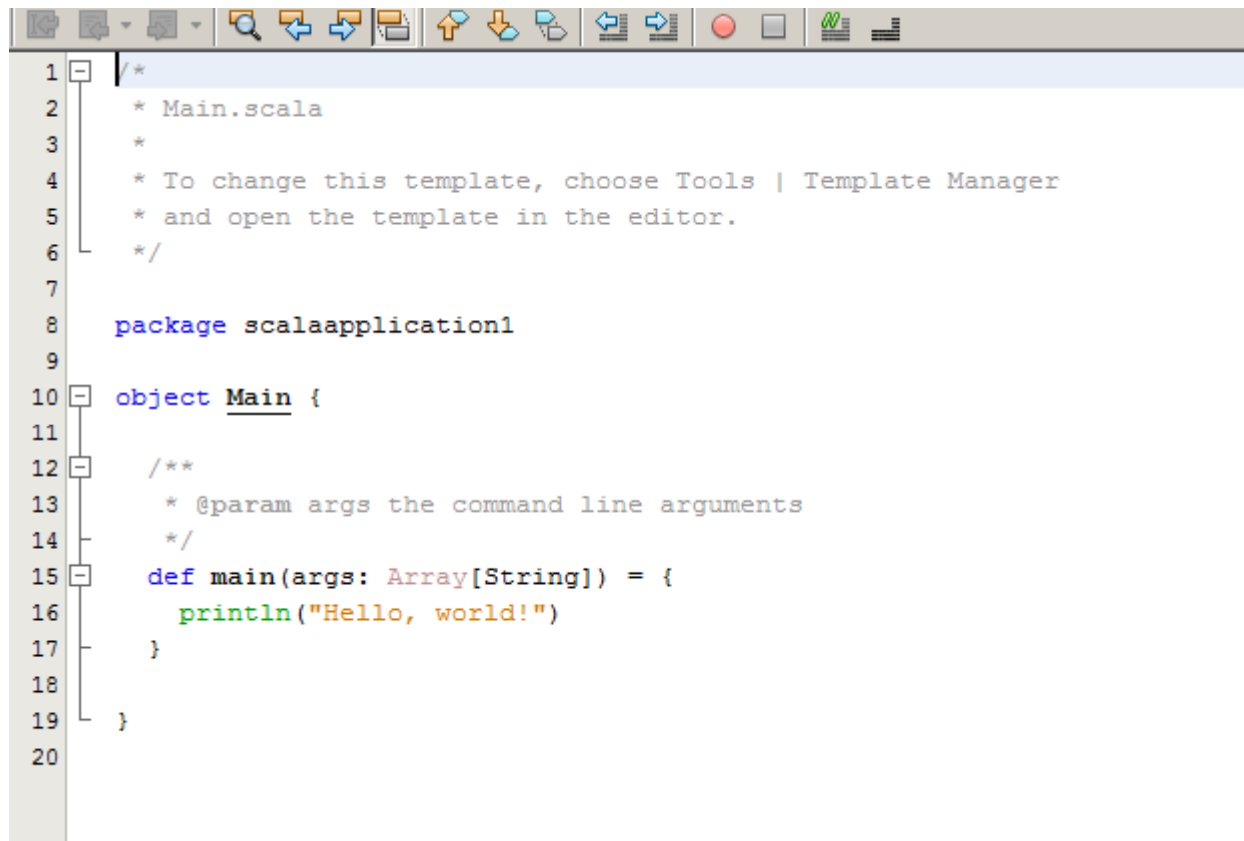
☒ Create Main Class

☒ Set as Main Project

< Back   Next >   **Finish**   Cancel   Help

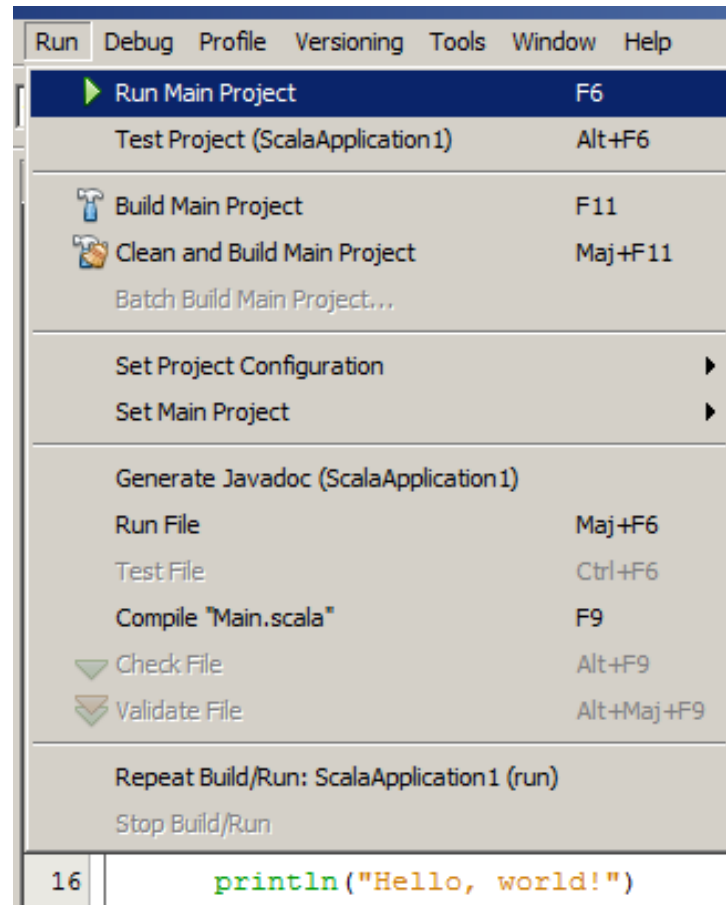
# Création d'un nouveau projet Scala (suite)

---



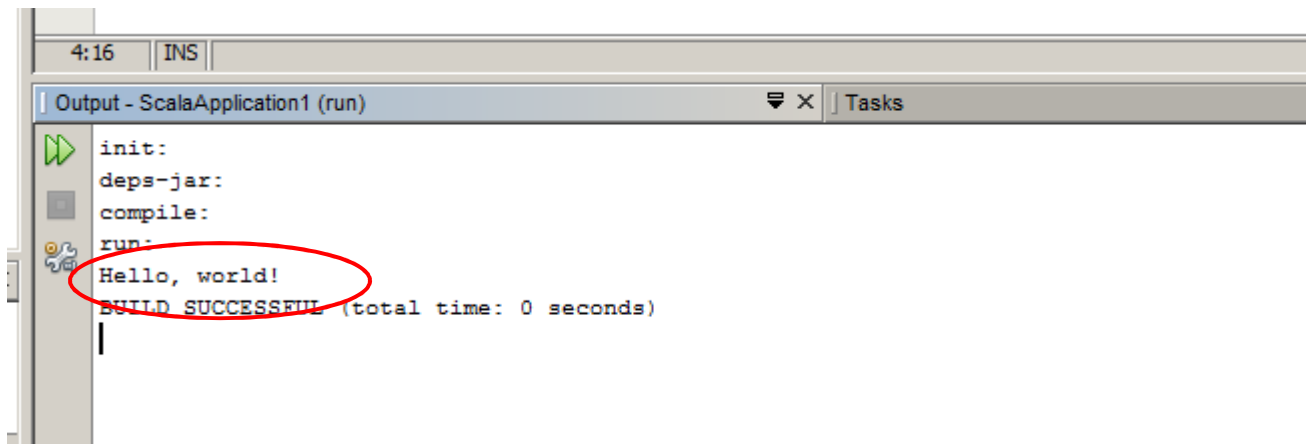
```
1  /*
2   * Main.scala
3   *
4   * To change this template, choose Tools | Template Manager
5   * and open the template in the editor.
6   */
7
8  package scalaapplication1
9
10 object Main {
11
12     /**
13      * @param args the command line arguments
14      */
15     def main(args: Array[String]) = {
16         println("Hello, world!")
17     }
18
19 }
20
```

# Création d'un nouveau projet Scala (suite)



# Création d'un nouveau projet Scala (suite)

---



The screenshot shows an IDE's output console for a project named 'ScalaApplication1 (run)'. The console displays the following sequence of events: 'init:', 'deps-jar:', 'compile:', and 'run:'. The output of the 'run' task is 'Hello, world!', which is circled in red. Below this, the console shows 'BUILD SUCCESSFUL (total time: 0 seconds)'. The IDE interface includes a status bar at the top showing '4:16' and 'INS', and a 'Tasks' tab on the right.

```
4:16 | INS |
Output - ScalaApplication1 (run)
init:
deps-jar:
compile:
run:
Hello, world!
BUILD SUCCESSFUL (total time: 0 seconds)
```

# Références

---

- ▶ Livre : *Programming Scala. A comprehensive step-by-step guide*. Martin Odersky, Lex Spoon, and Bill Venners. 2008.
- ▶ <http://www.scala-lang.org/>
- ▶ <http://scala.sygneca.com>